

Team Project Kickoff

Working together, reviewing each other's code, and shipping as a team

Agenda

- Merging to `main` – the rules
- What makes a good pull request
- How to review code effectively
- How to split up work as a team

Merging to `main`

The Merge Rules

Every PR to `main` needs **two approvals** before it can be merged.

Author opens PR

↓

Teammate reviews and approves ← must happen first

↓

Trainer / Mentor reviews and approves

↓

Author merges

Why teammate first? You are responsible for your team's codebase – your trainer/mentor approval is a quality gate, not a substitute for team ownership.

Merge Rules – What This Means in Practice

As the author

- Open your PR early – don't wait until the last minute
- Address all feedback before requesting the next reviewer
- You merge your own PR once approved
- Remember to merge main into your PR branch if other changes go in while your PR is reviewed

As a reviewer

- Review promptly – a blocked PR blocks your teammate's progress
- Be specific and constructive in your comments
- Approve when you are satisfied, not just when you want to move on
- If you have questions, ask them – don't just approve and hope

Writing a Good PR

What Every PR Must Include

A PR without these is not ready for trainer/mentor review.

Required	Why it matters
Ticket reference / link	Connects code to the requirement – reviewer knows what problem is being solved
Description of changes	Explains <i>what</i> changed and <i>why</i> – the diff shows what, not why
Testing done	Confirms it works and tells the reviewer how to verify it

Use the PR template in the repository – it prompts you for all of these.

PR Description – Good vs Bad

✗ Not enough information

Fixed the bug

✓ Gives the reviewer what they need

Fixes #42 – Expense form rejects negative amounts

Added Zod validation to the POST /expenses route to reject amounts less than or equal to zero. Returns a 400 with a field-level error message.

Testing done:

- Unit tests added for the validation schema (all pass)
- Manually tested via the form – negative values now show an error
- Manually tested the happy path – valid submissions still work

Code Reviews

Why Code Review Matters

Code review is not about finding fault – it is about shared ownership and learning.

- **Catch bugs** before they reach `main`
- **Share knowledge** – everyone understands more of the codebase
- **Keep quality consistent** – standards applied across the whole team
- **Learn from each other** – different people spot different things

The code that goes out is the *team's* code, not just the author's.

What to Look For

Use the self-review checklist in the PR template as your guide.

Area	Key question
Design & Structure	Are the layers doing only their own job?
Naming	Could you tell what each thing does from its name?
TypeScript	Are types strict and meaningful throughout?
Validation	Is all input validated server-side before it's trusted?
Error handling	Are status codes correct? Are errors handled gracefully?
Security	Could any input reach the database or the page unescaped?
Testing	Does every new behaviour have a test?

Review Anti-Patterns to Avoid

-  **Rubber-stamping** – approving without reading because it looks fine at a glance
-  **Nitpicking style** – flagging things the linter already catches
-  **Rewriting their code** – point to the issue, let them fix it
-  **Overwhelming comments** – focus on the 3-5 most impactful issues
-  **Being vague** – "this could be better" is not actionable

-  Ask questions you genuinely want answered
-  Acknowledge good decisions, not just problems
-  Be specific about what you'd like the author to reconsider

Splitting Up Work

Option A – End-to-End Ownership

One person takes a ticket and implements it fully: backend route, service, database, frontend view.

Advantages

- Simple to organise – grab a ticket and go
- No handoff needed between teammates
- One person understands the full stack for that feature

Watch out for

- Each ticket is a larger chunk of work – keep MVP in mind
- Higher chance of merge conflicts if teammates touch related files
- Can be harder to unblock yourself when stuck across the full stack

Option B – Split by Responsibility

Divide the team by layer: one or two people focus on backend, one or two on frontend.

Advantages

- Fewer git conflicts – layers rarely overlap
- Faster to build one layer at a time
- Builds deeper expertise in that area

Watch out for

- Requires clear API contracts agreed upfront
- Need to coordinate who's doing what on each ticket
- Can create bottlenecks if one layer gets ahead of the other
- **If you do this, rotate responsibilities** – everyone should experience the full stack

Option C – Pair Programming

Two people work on the same task together: one drives (writes code), one navigates (reviews and thinks ahead). Swap regularly.

Good for

- Project setup and configuration tasks
- Complex bugs or features that need extra brainpower
- Onboarding – get everyone familiar with the codebase faster
- The paired review counts as the teammate approval – the only valid case for an instant approval

Watch out for

- Both people's time goes to one task – less total throughput (be be just as efficient in the long run at times)
- **Rotate the driver seat regularly** – don't let one person dominate
- Aim to keep everyone on the team progressing on something – use pairing selectively, not as the default

Pair programming is absolutely fine for this academy – just be intentional about when and why you use it.

There Is No Single Right Answer

Both approaches work. What matters is that your team **agrees on an approach and communicates clearly**.

Whichever you choose, ask yourselves:

- How are we deciding who picks up the next ticket?
- How do we avoid two people working on the same file at the same time?
- What do we do if a ticket is blocked?
- Are we rotating responsibilities so everyone learns the whole stack?

Discuss this as a team now – before you start writing code.

Practical Tips for Team Git

- **Branch from an up-to-date `main`** – before starting any ticket, pull the latest
- **Keep branches short-lived** – the longer a branch lives, the worse the merge
- **Communicate before touching shared files** – routes, `app.ts`, schema files, config
- **Merge `main` into your branch regularly** – don't wait until your PR is ready
- **Small, focused PRs** – easier to review, less likely to conflict

A PR that takes 10 minutes to review gets reviewed. A PR that takes an hour gets deferred.

Summary

Topic	Key point
Merging to main	Teammate approval first, then trainer/mentor
Pull requests	Always include ticket link, description, and testing
Code reviews	Use the checklist; be specific and constructive
Splitting work	End-to-end, layer split, or pair programming – agree as a team and rotate
Git hygiene	Short branches, pull often, communicate before touching shared files

Over to You

Decide as a team:

1. How will you split up the work?
2. Who reviews whose PRs?
3. What shared files might cause conflicts – and how will you coordinate them?